RE Academy — Master Project Document

Living Document · Version 2.1 · Updated: 2026-06-11

> *This document is the single source of truth for everything about this project: what we planned, what we built, every problem we hit, every fix we applied, and every rule we learned. It is updated as we build. When starting a new session, read this first.*

Table of Contents

---

1. What We Are Building

A hybrid Learning Management System (LMS) and creator marketplace built specifically for real estate professionals. The platform serves three user types:

- **Learners** — real estate agents who want to consume educational content
- **Creators** — agents who want to build and sell their own courses
- **Admins** — internal company staff who manage the platform

Think of it as a combination of Kajabi, Thinkific, and Udemy — but built for real estate, owned by us.

We are also building a Template Marketplace (added during development) — a place where creators can sell downloadable resources (PDFs, Canva templates, Google Slides, checklists, etc.) separately from courses.

---

2. Technical Stack — Actual

> ***Important:** The original plan specified Next.js 14. The actual installed version is **Next.js 16**. Several conventions changed between 14 and 16 — see Section 7 for the issues this caused.*

Layer	Planned	Actual	Notes
Framework	Next.js 14 (App Router)	**Next.js 16.2.7** (Turbopack)	App Router used throughout

Language	TypeScript	TypeScript ✓	
Styling	Tailwind CSS + shadcn/ui	**Tailwind CSS v4** + shadcn/ui	v4 uses oklch colors, different syntax
Database	PostgreSQL via Supabase	PostgreSQL via Supabase ✓	
ORM	(not specified)	**Prisma**	`npx prisma db push` — NEVER `migrate dev`
Auth	Supabase Auth	Supabase Auth ✓	SSR package `@supabase/ssr`
Video	Mux	Mux ✓	`<mux-player>` web component
File Storage	Cloudflare R2	Cloudflare R2 ✓	
Payments	Stripe + Stripe Connect	Stripe + Stripe Connect ✓	
Email	Resend	Resend (configured, not fully wired)	
Hosting	Vercel	Vercel (target)	Dev: localhost:3000
Route Protection	Middleware	**proxy.ts** (Next.js 16 renamed this)	See Section 7, Problem #6
Error Monitoring	Sentry	Not yet implemented	Phase 0.10 — still pending

Dev Commands

```
# Start dev server
cd /Users/alyssaspecht/real-estate-edu && npm run dev

# Kill and restart dev server
pkill -f "next dev" && sleep 2 && npm run dev

# Database schema push (ALWAYS use this, never migrate dev)
npx prisma db push

# Check server logs
tail -f /tmp/nextdev.log
```

---

3. User Roles & Permissions

Permission	Learner	Creator	Admin
Browse and search courses	✓	✓	✓
Purchase courses	✓	✓	✓

Access enrolled content	✓	✓	✓
Create and manage courses	—	✓	✓
View own sales and students	—	✓	✓
Apply to become a creator	✓	—	—
Approve creator applications	—	—	✓
Manage all users	—	—	✓
Manage all content	—	—	✓
View platform-wide reporting	—	—	✓
Feature and promote content	—	—	✓
Issue refunds	—	—	✓
Upload templates to marketplace	—	✓	✓
Manage template categories	—	—	✓

Route protection is enforced in `proxy.ts` (formerly `middleware.ts` in Next.js <16):

- `/dashboard`, `/creator`, `/admin` → redirect to `/login` if not authenticated
- `/login`, `/signup` → redirect to `/dashboard` if already authenticated

---

4. Implementation Status by Phase

Phase 0 — Infrastructure & Foundation ✅ COMPLETE

#	Feature	Status	Notes
0.1	Project Setup	✅ Done	Next.js 16, TypeScript, Tailwind v4
0.2	Database Schema	✅ Done	Prisma + Supabase PostgreSQL
0.3	Authentication System	✅ Done	Email/password + Google OAuth
0.4	Role-Based Access	✅ Done	proxy.ts with route protection
0.5	File Upload Pipeline	✅ Done	Cloudflare R2
0.6	Video Pipeline	✅ Done	Mux upload + `<mux-player>`
0.7	Payment Foundation	✅ Done	Stripe + Stripe Connect
0.8	Email System	⚠️ Partial	Resend configured, templates not fully wired
0.9	CI/CD Pipeline	⏳ Pending	Vercel deployment set up, no automated tests yet
0.10	Error Monitoring	⏳ Pending	Sentry not yet installed

Phase 1 — Internal LMS ✅ LARGELY COMPLETE

#	Feature	Status	Notes
1.1	Admin Course Creation	✅ Done	
1.2	Module & Lesson Builder	✅ Done	Reorder via position field
1.3	Video Lesson Upload	✅ Done	Mux upload + playback
1.4	Text Lesson Content	✅ Done	
1.5	PDF & Resource Attachments	✅ Done	
1.6	Manual Learner Enrollment	✅ Done	Admin-triggered
1.7	Lesson Player	✅ Done	`LessonPlayer.tsx` with mux-player
1.8	Progress Tracking	✅ Done	API at `/api/progress`
1.9	Resume Playback	✅ Done	Timestamp stored per lesson
1.10	Course Completion	✅ Done	Triggered when all lessons complete
1.11	Admin User Management	✅ Done	
1.12	Admin Course Management	✅ Done	

Phase 2 — Creator Marketplace ✅ LARGELY COMPLETE

#	Feature	Status	Notes
2.1	Creator Application	✅ Done	
2.2	Creator Approval (Admin)	✅ Done	
2.3	Stripe Creator Onboarding	✅ Done	Stripe Connect KYC
2.4	Creator Dashboard	✅ Done	
2.5	Creator Course Builder	✅ Done	Scoped to own courses
2.6	Course Pricing	✅ Done	Free or paid in cents
2.7	Course Publishing Controls	✅ Done	
2.8	Creator Public Profile	✅ Done	`/creators/[id]`
2.9	Course Browse Page	✅ Done	Category filters
2.10	Course Detail Page	✅ Done	Full info + curriculum preview
2.11	Free Preview Lessons	✅ Done	`isFreePreview` flag
2.12	Checkout & Payment	✅ Done	Stripe Checkout
2.13	Enrollment on Purchase	✅ Done	Webhook grants access
2.14	Purchase History	✅ Done	

2.15	Creator Student List	✅ Done	
2.16	Creator Sales Dashboard	✅ Done	
2.17	Creator Payouts	✅ Done	Stripe Connect
2.18	Basic Course Search	✅ Done	
2.19	Email Notifications	⚠️ Partial	Resend set up, not all triggers wired
2.20	Admin Marketplace Oversight	✅ Done	

Phase 3 — Growth & Discovery ⚠️ IN PROGRESS

#	Feature	Status	Notes
3.1	Course Reviews & Ratings	✅ Done	`ReviewSection.tsx`
3.2	Instructor Ratings	✅ Done	Aggregate on creator profile
3.3	Advanced Search & Filters	⏳ Pending	
3.4	Featured & Promoted Content	⏳ Pending	
3.5	Completion Certificates	⏳ Pending	
3.6	Learning Paths / Bundles	✅ Done	`/paths` + PathEnrollButton
3.7	Coupon & Promo Codes	⏳ Pending	
3.8	Creator Video Analytics	⏳ Pending	
3.9	Creator Enrollment Trends	⏳ Pending	
3.10	Basic Referral Program	⏳ Pending	

Phase 4 — Community & AI ⚠️ PARTIAL (some built early)

#	Feature	Status	Notes
4.1	Course Discussion Boards	✅ Done	`CourseDiscussion.tsx`, per-course, enrolled only
4.2	Creator Community Spaces	⏳ Pending	
4.3	Events & Challenges	⏳ Pending	
4.4	**Gamification**	✅ Done	**Built during Phase 2 session** — see below
4.5	AI Lesson Summaries	⏳ Pending	Blocked: video lessons have no transcript yet

4.6	AI Course Builder	⏳ Pending	
4.7	Badges (merged into 4.4)	✅ Done	10 badges implemented
4.8	Leaderboards (merged into 4.4)	✅ Done	Top 10 + current user rank
4.9	Live Session Integration	⏳ Pending	
4.10	Native Mobile App	⏳ Pending	
4.11	B2B / Team Accounts	⏳ Pending	

Feature 4.4 — Gamification Details (Implemented 2026-05-02)

- **XP System:** Points awarded for lesson complete (+10), course complete (+100), streak bonuses, template download (+5)
- **Streak System:** Daily activity tracking, bonus XP at 3/7/30 day streaks
- **10 Badges:** first-lesson 🎯, first-course 🎓, three-courses 📚, streak-3 🔥, streak-7 ⚡, streak-30 💎, xp-100 ⭐, xp-500 🚀, xp-1000 🏆, first-template 🛠️
- **XP Levels:** Newcomer → Rising Agent → Licensed Pro → Senior Agent → Top Producer → Legend
- **Leaderboard:** Top 10 users by XP with medals, current user rank shown if outside top 10
- **XPToast:** Animated toast notification after completing a lesson
- **Key files:** `lib/gamification.ts`, `components/gamification/`, `app/dashboard/progress/page.tsx`

Template Marketplace — BONUS FEATURE (Built 2026-05-02)

This was not in the original plan. Added as a high-value extension.

Feature	Status	Notes
Template browse page	✅ Done	`/templates` with category filter
Template detail page	✅ Done	`/templates/[slug]`
Template upload (creator)	✅ Done	`TemplateForm.tsx`
File delivery (download)	✅ Done	Cloudflare R2 signed URLs
Link delivery (Canva/Google Slides)	✅ Done	`deliveryType: "LINK"`, stored privately, revealed post-purchase
Template categories (admin-controlled)	✅ Done	12 categories seeded, creator can suggest new ones

Free template purchase	✅ Done	Instant access, no payment
Paid template purchase	✅ Done	Stripe Checkout → access

Template Categories (as of 2026-06-08):

1. 📋 Listing Presentations
2. 🏡 Buyer Guides
3. 📊 Market Reports
4. 📄 Contract Templates
5. 💼 Business Planning
6. 📱 Social Media Templates *(added)*
7. ✉️ Email & Follow-Up *(added)*
8. ✅ Checklists & Systems *(added)*
9. 🗺️ Local Guides & Farming *(added)*

Key Design Decision — Template Delivery:

- Creators upload either a **file** (PDF, PPTX, etc. → stored in R2) or a **link** (Canva "use template" URL, Google Slides, Notion, etc.)
- Link templates: the URL is stored privately in `templateLinkUrl` and only revealed after purchase via `/api/templates/[slug]/download`
- This mirrors how Etsy handles Canva template sales

---

5. Architecture & Key Technical Decisions

Auth Flow (Critical — Read Before Touching Anything Auth-Related)

```
Browser Request
     ↓
proxy.ts (runs on EVERY request via Next.js proxy convention)
  - Creates Supabase server client
  - Calls supabase.auth.getUser() → refreshes access token if expired
  - Sets updated cookies on BOTH request and response
  - Redirects /login → /dashboard if user is authenticated
  - Redirects /dashboard, /creator, /admin → /login if not authenticated
     ↓
Server Component (layout.tsx, page.tsx)
  - Calls getCurrentUser() → supabase.auth.getUser() + prisma.user.findUnique()
  - Returns null if Supabase auth valid but no DB user (important edge case)
     ↓
Client Component hydration
  - TopNav: if server said "logged out" but client has a session → router.refresh()
  - Login/Signup pages: useEffect checks getSession() → redirect if already logged in
```

Why this three-layer approach: Without it, stale/expired tokens cause a mismatch where the server renders the nav as "logged out" while the browser actually has a valid session. Clicking "Sign In" would appear to do nothing (it navigated to /login → proxy redirected to /dashboard instantly).

Supabase SSR Pattern

```
// Server client (app/api/*, server components, proxy.ts)
import { createClient } from '@/lib/supabase/server'
const supabase = await createClient()

// Client (components with 'use client')
import { createClient } from '@/lib/supabase/client'
const supabase = createClient()
```

Database Operations

```
# Modify schema in prisma/schema.prisma, then:
npx prisma db push          # ALWAYS — pushes schema to Supabase
npx prisma generate         # Regenerates types (usually runs automatically)

# NEVER run:
npx prisma migrate dev      # Creates migration files — unnecessary complexity for
this project
```

Next.js 16 Conventions (Different from Next.js 14 docs)

Next.js 14	Next.js 16
`middleware.ts`	`proxy.ts` (with `export async function proxy()`)
`export const config = { matcher: [...] }`	Same, still works
`NextResponse.next()`	Same

---

6. UI Design System

Design Direction

Dark navy glassmorphism — inspired by Apple visionOS liquid glass. Deep navy background with animated color orbs, frosted glass cards, electric blue accents.

Implemented: **2026-05-01** (comprehensive UI pass during Phase 2 work)

Color System (Tailwind v4 oklch)

```
/* Dark mode (default) */
--background: oklch(0.11 0.022 255)    /* Deep navy */
--foreground: oklch(0.94 0.008 240)    /* Near white */
--primary: oklch(0.65 0.2 255)         /* Electric blue */
--card: oklch(1 0 0 / 5%)             /* Barely-there white */
--border: oklch(1 0 0 / 9%)           /* Subtle white border */
--orb-opacity: 0.45                    /* Background orb visibility */
```

CSS Utility Classes

Class	Use For	Has Hover?
`.glass-card`	Content cards, feature cards, info panels	Yes (brighter border, no transform)
`.glass-panel`	**Forms, containers, anything with inputs**	No — safe for interactive containers
`.glass`	Nav overlays, lightweight glass	No
`.nav-blur`	The top navigation bar	No
`.glow-blue`	Primary CTA buttons	Yes (stronger glow, no transform)
`.gradient-text`	Hero headings	No
`.shimmer-border`	Decorative cards (adds shimmer on hover)	Yes (shimmer sweep)
`.text-glow`	Headings that need extra punch	No

CRITICAL CSS RULE — Never Use transform on Interactive Elements

`transform: translateY()` on hover states shifts the element's position. When the element moves, the cursor is no longer over it, so the click misses. This caused multiple sessions of "buttons don't work" debugging.

Rule: NEVER add `transform` to `.glass-card:hover`, `.glow-blue:hover`, or any element that contains clickable content.

MeshBackground Component

Located: `components/MeshBackground.tsx`

- 4 animated orbs with CSS keyframe animations
- `pointer-events: none` on the entire container (critical)
- `fixed inset-0 z-0` — sits behind everything
- Content wrapper: `relative z-10` — sits above mesh

Tailwind v4 Non-Standard Values

Some Tailwind opacity values that look valid are NOT in v4's default config. These cause silent failures (class applied, no style rendered):

Don't Use	Use Instead
`bg-white/12`	`bg-white/10`
`bg-white/3`	`bg-white/5`
`border-white/8`	`border-white/10`
`border-white/6`	`border-white/5`

Use oklch values in `globals.css` for custom opacities.

---

7. Problem Log — Issues & How We Fixed Them

This is the most important section. Every problem we hit is recorded here so we never debug the same thing twice.

Problem 1: Mesh Background Orbs Not Visible

Date: ~2026-05-01 **Symptom:** Background was solid dark navy. The animated color orbs from MeshBackground were invisible. **Root Cause:** Three compounding issues:

1. `--orb-opacity: 0.22` — too low
2. `filter: blur(80px)` — too heavy, washing out color
3. Orbs were too small (40vw max)

Fix Applied:

```
--orb-opacity: 0.45;              /* Was: 0.22 */
filter: blur(60px);               /* Was: blur(80px) */
/* Orb 1: */ width: 80vw; max-width: 1100px;   /* Was: ~500px */
/* Orb 2: */ width: 70vw; max-width: 950px;    /* Was: ~400px */
```

Prevention: When orbs are invisible, check opacity first, then blur, then size. All three must be right.

Problem 2: Non-Standard Tailwind Opacity Values (Silent Failure)

Date: ~2026-05-01 **Symptom:** Glass styling looked wrong in some areas. No errors — just ignored styles. **Root Cause:** Tailwind v4 does not include every possible opacity step. Values like `white/12`, `white/3`, `white/8`, `white/6` are not in the default config and produce no output. **Fix:** Replaced all non-standard values:

- `white/12` → `white/10`
- `white/3` → `white/5`
- `white/8` → `white/10`
- `white/6` → `white/5`

Prevention: Stick to standard Tailwind opacity steps: 5, 10, 15, 20, 25, 30, 40, 50, 60, 70, 75, 80, 90, 95. For anything in between, use CSS custom properties in `globals.css`.

Problem 3: Buttons Not Clickable After Adding Hover Animations

Date: ~2026-05-01 **Symptom:** After adding glass card styles, buttons on login/signup/nav were completely unclickable. Clicking appeared to do nothing. **Root Cause:** `transform: translateY(-2px)` on `.glass-card:hover` and `transform: translateY(-1px)` on `.glow-blue:hover`. When these elements lift on hover, the cursor is no longer directly over the element — the click registers on whatever is behind them. **Fix Applied:**

- Removed ALL `transform` from `.glass-card:hover`
- Removed ALL `transform` from `.glow-blue:hover`
- Removed `transform` from the `transition` property too
- Created `.glass-panel` class (identical to `.glass-card` but with NO hover state at all) for use on form containers

Prevention Rule (Permanent): NEVER use `transform: translateY()` on hover for any element that contains or IS a clickable element. Hover effects on interactive elements must only use `box-shadow`, `border-color`, `background`, `opacity`, `filter`. No positional transforms.

Problem 4: Dev Server ERR_CONNECTION_REFUSED

Date: ~2026-05-01 **Symptom:** `localhost:3000` refused connection. Site completely unreachable. **Root Cause:** Next.js dev server process crashed or was never started. **Fix:**

```
pkill -f "next dev"   # Kill any zombie process
sleep 2
cd /Users/alyssaspecht/real-estate-edu && npm run dev
```

Check if running: `lsof -i :3000 | grep node`

Prevention: Before debugging any UI issue, verify the server is actually running. Many "broken" issues are just a dead server.

Problem 5: Chrome Extension Errors in Console ("message channel closed")

Date: ~2026-05-01 **Symptom:** Console flooded with: *"A listener indicated an asynchronous response by returning true, but the message channel closed before a response was received"* **Root Cause:** A Chrome browser extension (likely a tab manager or password manager) was injecting scripts that conflicted with the page's JavaScript message channels. This was NOT our code. **Fix:** Test in Chrome incognito mode (extensions disabled) or in a different browser.

Identification: These errors come from extension scripts, not from `localhost`. If the error source URL contains `chrome-extension://` or is from `node_modules/next`, it's an extension conflict.

Prevention: Always test in incognito when debugging click/interaction issues. Extensions can intercept events and prevent them from reaching page handlers.

Problem 6: `middleware.ts` Conflict — Routes Return 404 (Next.js 16)

Date: 2026-06-08 **Symptom:** After creating `middleware.ts`, ALL routes returned 404. Site completely broken. **Root Cause:** Next.js 16 renamed the middleware convention from `middleware.ts` → `proxy.ts`. A `proxy.ts` already existed with the correct Supabase session refresh logic. Creating a new `middleware.ts` caused a conflict that broke all routing.

Error in logs:

```
⚠ The "middleware" file convention is deprecated. Please use "proxy" instead.
Unhandled Rejection: Error: Both middleware file "./middleware.ts" and proxy file
"./proxy.ts" are detected. Please use "./proxy.ts" only.
```

Fix: Delete `middleware.ts`. Keep only `proxy.ts`.

Prevention Rule (Permanent): In this project, route interception logic lives ONLY in `proxy.ts`. Never create `middleware.ts`. The function export must be `export async function proxy()`, not `export async function middleware()`.

Problem 7: "Sign In / Get Started Don't Work" — Auth State Mismatch

Date: 2026-06-08 (multiple sessions of debugging) **Symptom:** User clicks "Sign in" or "Get started" nav links. Nothing appears to happen. **Root Cause (Multi-part):**

1. Supabase access tokens expire every ~1 hour. When expired, the server renders the nav as "logged out" even if the browser has a valid refresh token.
2. When the user clicks "Sign in" → navigates to `/login` → proxy.ts detects valid session → immediately redirects to `/dashboard`. Looks like nothing happened.
3. The nav was stuck in server-rendered "logged out" state and never updated.

Fix Applied (Three layers):

Layer 1 — proxy.ts (already existed):

```
// Calls getUser() on every request → refreshes token → sets updated cookies
await supabase.auth.getUser()
// Redirects authenticated users away from /login and /signup
```

Layer 2 — TopNav client-side sync:

```
useEffect(() => {
  if (serverUserRole) return // server already correct
  const supabase = createClient()
  supabase.auth.getSession().then(({ data: { session } }) => {
    if (session) router.refresh() // trigger server re-render with correct auth
state
  })
}, [serverUserRole, router])
```

Layer 3 — Login/Signup pages:

```
useEffect(() => {
  const supabase = createClient()
  supabase.auth.getSession().then(({ data: { session } }) => {
    if (session) router.replace('/dashboard')
  })
}, [router])
```

Prevention: Any time auth-related code is modified, re-test: (1) fresh visit while logged out, (2) clicking Sign In, (3) submitting the login form, (4) landing on dashboard. The three-layer approach makes auth state resilient to token expiry.

Problem 8: Missing User Record Creation on Signup/Login — Empty Dashboard

Date: 2026-06-08 **Symptom:** User signs up, logs in, and dashboard shows "No courses yet" / 0 stats / empty progress, even after enrolling. Supabase Table Editor showed every table at 0 rows except `Badge` (which is seeded automatically). **Root Cause:** Supabase Auth manages its own `auth.users` table — completely separate from our application's Prisma `User` table. Nothing in the codebase ever created a corresponding Prisma `User` row when someone signed up or logged in via OAuth/magic link. Verified via:

```
grep -r "prisma.user.create\|prisma.user.upsert" .
# → no results anywhere in the codebase
```

Every query that depended on `user.id` (enrollments, progress, XP, streaks) silently returned nothing because the `User` row it should have joined against never existed.

Fix Applied: Converted `getCurrentUser()` (`lib/auth/getUser.ts`) — which is called on every protected page — into a self-healing upsert:

```
const dbUser = await prisma.user.upsert({
  where: { id: user.id },
  update: {
    email: user.email ?? '',
  },
  create: {
    id: user.id,
    email: user.email ?? '',
    name: user.user_metadata?.name ?? user.email?.split('@')[0] ?? 'User',
    role: 'LEARNER',
  },
})
```

This guarantees a Prisma `User` row exists for any signed-in Supabase user, regardless of how they signed up (email/password, OAuth, magic link) — past, present, or future.

Prevention Rule (Permanent): Every page/route that needs the current user MUST go through `getCurrentUser()` — never call `supabase.auth.getUser()` directly and assume a matching Prisma `User` row exists. If a new auth entry point is added (new OAuth provider, invite flow, admin-created accounts, etc.), it must still route through `getCurrentUser()` before any Prisma query that references `userId` .

---

8. Rules We Have Learned (Never Break These)

These are permanent rules derived from real problems we hit. They are not suggestions.

Auth & Routing

1. **Route protection lives in `proxy.ts` only.** Never in `middleware.ts` (doesn't exist in Next.js 16). Never in `layout.tsx` .
2. **Never call `prisma.migrate dev` .** Always use `npx prisma db push` .
3. **`getCurrentUser()` can return null even with a valid Supabase session** if the user record doesn't exist in Prisma. Handle this case everywhere.
4. **After any auth-related change, always re-test:** sign in, sign up, protected route access, and the nav state.
5. **A Supabase auth user is NOT a Prisma `User` row.** Always read the current user via `getCurrentUser()` (which upserts the Prisma row) — never `supabase.auth.getUser()` directly when you need `user.id` for a Prisma query.

CSS & UI

6. **Never use `transform: translateY()` on hover for interactive elements.** Use box-shadow and border-color only.
7. **Use `.glass-panel` for forms and containers.** Use `.glass-card` for non-interactive display cards only.
8. **Tailwind v4 opacity steps:** Only use 5, 10, 15, 20, 25, 30, 40, 50, 60, 70, 75, 80, 90, 95. Anything else silently fails.
9. **`shimmer-border::before` must have `pointer-events: none`** — already in CSS, never remove it.
10. **`MeshBackground` must keep `pointer-events: none`** on its wrapper div — already set, never remove it.

Development Process

11. **Before debugging UI interaction issues, verify the dev server is actually running.**
12. **Test in incognito mode** when debugging click issues — Chrome extensions can intercept events.
13. **Check the server logs** (`tail -f /tmp/nextdev.log`) before concluding something is broken. The answer is usually there.
14. **After any CSS change, test that buttons still click** — especially login, signup, and any form submit.

Database

15. **Never use `prisma.migrate dev` — always `prisma db push`.**
16. **Seed data with upsert** (idempotent), not insert — so running seeds multiple times doesn't break anything.

---

9. Database Schema Overview

Key models and their relationships. Full schema at `prisma/schema.prisma`.

```
User
  ├── role: LEARNER | CREATOR | ADMIN
  ├── xp: Int (gamification)
  ├── streak: UserStreak?
  ├── badges: UserBadge[]
  ├── enrollments: Enrollment[]
  ├── purchases: Purchase[]
  ├── progress: LessonProgress[]
  ├── reviews: Review[]
  └── profile: CreatorProfile?

Course
  ├── status: DRAFT | PUBLISHED
  ├── price: Int (cents)
  ├── modules: Module[]
  │     └── lessons: Lesson[]
  │           └── progress: LessonProgress[]
  ├── enrollments: Enrollment[]
  ├── reviews: Review[]
  └── creator: User

Template
```

```
  ├── deliveryType: "FILE" | "LINK"
  ├── fileUrl: String?        (Cloudflare R2 for FILE type)
  ├── templateLinkUrl: String? (private URL for LINK type — Canva etc.)
  ├── categoryNote: String?   (creator suggestion for new category)
  ├── category: TemplateCategory
  └── purchases: TemplatePurchase[]

Gamification Models:
  UserStreak  (userId, current, longest, lastActivityDate)
  Badge       (slug, name, description, icon, category, threshold)
  UserBadge   (userId + badgeId unique)

Payment Models:
  Order       (Stripe order record)
  Enrollment  (userId + courseId — grants access)
  TemplatePurchase (userId + templateId — grants access)
```

---

10. File & Folder Map

```
/real-estate-edu
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx          ← Email/password login form
│   │   └── signup/page.tsx         ← Sign up form
│   ├── admin/                      ← Admin dashboard and tools
│   ├── api/
│   │   ├── auth/signout/           ← Sign out endpoint
│   │   ├── gamification/
│   │   │   ├── leaderboard/        ← GET top 10 by XP
│   │   │   └── me/                 ← GET current user XP/badges/streak
│   │   ├── progress/route.ts       ← POST lesson progress + triggers gamification
│   │   ├── stripe/                 ← Stripe checkout + webhooks
│   │   └── templates/[slug]/
│   │       ├── download/           ← Returns file URL or Canva link post-purchase
│   │       └── purchase/           ← Free template instant purchase
│   ├── auth/callback/route.ts      ← Supabase OAuth callback
│   ├── creator/                    ← Creator dashboard
│   ├── courses/
│   │   └── [slug]/
│   │       └── lessons/[lessonId]/ ← Lesson player page
│   ├── dashboard/
│   │   ├── page.tsx                ← Learner dashboard with gamification strip
│   │   └── progress/page.tsx       ← XP + badges + streaks + leaderboard
│   ├── paths/                      ← Learning paths browse + detail
│   ├── templates/                  ← Template marketplace browse + detail
│   ├── globals.css                 ← ⚠️ CRITICAL — all glass/mesh/color CSS lives
here
│   ├── layout.tsx                  ← Root layout: MeshBackground + TopNav +
ThemeProvider
│   └── page.tsx                    ← Homepage
```

```
├── components/
│   ├── gamification/
│   │   ├── BadgeGrid.tsx
│   │   ├── Leaderboard.tsx
│   │   ├── StreakWidget.tsx
│   │   ├── XPBar.tsx
│   │   └── XPToast.tsx
│   ├── learner/
│   │   ├── CourseDiscussion.tsx
│   │   ├── EnrollButton.tsx
│   │   ├── LessonPlayer.tsx        ← Video player + progress tracking + XP toast
│   │   └── ReviewSection.tsx
│   ├── templates/
│   │   ├── TemplateDetail.tsx
│   │   ├── TemplateBrowser.tsx
│   │   └── TemplateForm.tsx        ← Upload form with FILE/LINK delivery toggle
│   ├── MeshBackground.tsx          ← Animated background orbs — pointer-events: none
│   ├── TopNav.tsx                  ← Nav with client-side auth sync
│   ├── ThemeProvider.tsx
│   └── ThemeToggle.tsx
├── lib/
│   ├── auth/getUser.ts             ← getCurrentUser() — Supabase + Prisma lookup
│   ├── gamification.ts             ← XP, streaks, badge engine
│   ├── prisma.ts                   ← Prisma client singleton
│   └── supabase/
│       ├── client.ts               ← Browser Supabase client
│       └── server.ts               ← Server Supabase client (SSR)
├── prisma/
│   └── schema.prisma               ← Source of truth for DB structure
├── proxy.ts                        ← ⚠️ Route protection + session refresh (Next.js
16)
└── .env                            ← All environment variables
```

---

11. Feature Roadmap (Full)

PHASE 0 — Infrastructure (Complete)

PHASE 1 — Internal LMS (Complete)

PHASE 2 — Creator Marketplace (Complete)

PHASE 3 — Growth & Discovery (In Progress)

#	Feature	Priority	Notes
3.3	Advanced Search & Filters	HIGH	Filter by price, category, rating, duration
3.4	Featured & Promoted Content	MEDIUM	Admin pins courses to homepage
3.5	Completion Certificates	MEDIUM	PDF generated + emailed via Resend
3.7	Coupon & Promo Codes	MEDIUM	Stripe coupon integration

3.8	Creator Video Analytics	LOW	Mux data API
3.9	Creator Enrollment Trends	LOW	Chart per course over time
3.10	Basic Referral Program	LOW	Defer — legal/tax complexity

PHASE 4 — Community & AI (Partially done, mostly pending)

#	Feature	Priority	Notes
4.2	Creator Community Spaces	MEDIUM	Per-creator groups
4.3	Events & Challenges	LOW	
4.5	AI Lesson Summaries	MEDIUM	**Blocked:** need video transcripts first
4.6	AI Course Builder	LOW	Upload recording → AI structures course
4.9	Live Session Integration	LOW	Zoom embed
4.10	Native Mobile App	LOW	React Native — long-term
4.11	B2B / Team Accounts	MEDIUM	Companies buy seats

PHASE 5 — AI Content Creation & Team Tools (Planned — added 2026-06-11)

Context: Our creators aren't a single persona — a "Creator" account may be a Team Lead (manages agents on their team), an Attractor/Recruiter (drives recruiting funnels), or a Coach (1:1/group coaching). Each has different content and team-management needs, but they share a common foundation: AI-assisted content creation and team/resource management. Phase 5 builds that shared foundation, then layers persona-specific tools on top.

#	Feature	Priority	Notes
5.1	AI Course/Lesson Builder from Source Material	✅ DONE (2026-06-11, text input)	Creator pastes a transcript/notes; AI (Opus 4.8, structured output) drafts a full course — title, description, modules, and lessons with rewritten content. Creator reviews/edits inline, then creates as a Draft course. Built at `/creator/courses/ai-builder` (`components/creator/AICourseBuilder.tsx`, `app/api/ai/course-builder/route.ts` + `/create/route.ts`). **Remaining:** PDF upload and voice input (5.2) not yet built — text paste only for now.
5.2	Voice-to-Course Creation	MEDIUM	Creator talks through course content via voice chat; AI transcribes + structures it into a course/lesson draft. Depends on 5.1's structuring pipeline.
5.3	Creator Personas / Role Types	HIGH	Add a "creator type" (Team Lead, Attractor, Coach, General) on the Creator profile. Drives which tools/dashboards are surfaced — foundation for 5.4–5.7.
5.4	Team Roster & Bulk Invite	✅ DONE (2026-	Creator/Admin can paste a list (one person per line, "Name, email" or just email) at `/creator/team`. Each line upserts a

		06-11)	`User` (role LEARNER if new) and creates a `TeamMembership` linking them to the team lead. Person doesn't need to sign up first — when they later log in via Supabase, `getCurrentUser()`'s existing upsert matches by email and links the account.
5.5	Assign Courses & Resources to Team Members	✅ DONE (2026-06-11, courses only)	On `/creator/team`, team lead picks member(s) + published course(s) and assigns in one click. Creates an `Assignment` row and auto-creates an `Enrollment` so progress tracks normally. Roster view shows each member's assigned courses with completed/total lesson counts. **Remaining:** resource-only assignment (outside of a course) not yet built — courses only for now.
5.5a	Course Visibility: Public vs. Private (Invite-Only)	✅ DONE (2026-06-11)	Added `visibility` field (`PUBLIC`/`PRIVATE`) to `Course`. Set at creation via a radio choice on the New Course form ("Public — listed for purchase" vs. "Private — invite only"). `/courses` catalog now filters to `status: PUBLISHED, visibility: PUBLIC` only — private courses never appear there. Private courses are only reachable via the team Assign flow (5.5) or a direct link.
5.6	Document Signing Platform	HIGH	Team Lead can send documents (e.g., team agreements) to team members for e-signature, track signed/unsigned status. Likely candidates: build lightweight in-house signing, or integrate a provider (e.g., Documenso, HelloSign/Dropbox Sign) — needs evaluation.
5.7	AI Document Drafting (e.g., Team Agreements)	MEDIUM	If a Team Lead doesn't have a document (like a team agreement), AI researches and drafts one as a starting point, which then flows into 5.6 for signing. Important: AI-drafted legal/contract documents should carry a disclaimer recommending legal review — not a substitute for an attorney.

Suggested build order: 5.3 (persona foundation) → 5.1 (AI content builder, highest standalone value) → 5.4/5.5 (team roster + assignment, depends on 5.3) → 5.6 (signing platform) → 5.7 (AI drafting, depends on 5.6 existing) → 5.2 (voice, nice-to-have layer on 5.1).

Design Principle: Radical Simplicity. These are non-technical users (team leads, coaches, attractors) — every Phase 5 feature must be usable with near-zero learning curve:

- Upload a file / paste text / talk → AI does the structuring. No manual form-building required as the default path.
- Bulk team upload should accept messy real-world input (CSV, pasted spreadsheet rows, even a pasted contact list) and let AI map columns — don't force a rigid template.
- Assigning courses/resources/agreements to a team should be a single flow: pick people → pick what to assign → done. Avoid multi-step wizards.
- AI-drafted documents (5.7) should produce a ready-to-send draft, not a blank template with instructions.

- Every AI-generated draft (course, lesson, document) must have a clear human-review/edit step before it goes live or gets sent — "AI drafts, human approves," never fully automatic for anything legal or team-facing.

Pending Infrastructure

- [ ] Email notifications — Resend templates for purchase receipts, enrollment confirmations, discussion replies
- [ ] AI assistant — needs video transcript ingestion (can't summarize what it can't read)
- [ ] Sentry error monitoring (Phase 0.10 — never implemented)
- [ ] Automated test suite (Phase 0.9 — CI/CD exists, no automated tests)

Features NOT Building (and Why)

Feature	Reason
Custom domains for creators	Complex routing/SSL, high support burden
Content DRM/piracy protection	Mux provides basic protection; over-engineering
Multi-language/localization	English market first
Cohort-based courses	Scheduling complexity
Built-in video conferencing	Use Zoom integrations
Real estate CE credit certification	State-specific regulations — separate initiative

---

12. Testing Protocol

The Golden Rule

We do not move to the next feature until the current feature passes all acceptance criteria.

User Story Format (Required Before Any Feature Starts)

```
FEATURE NAME

AS A [learner / creator / admin]
I WANT TO [do something specific]
SO THAT [I get some specific value]

ACCEPTANCE CRITERIA:
✓ Happy path works exactly as expected
✓ Edge cases are handled gracefully
✓ Error states show a clear message (never a blank screen or crash)
✓ Unauthorized users cannot access this feature
✓ Clicking "Sign in" and "Get started" still works (auth regression)
✓ Mobile experience works correctly

REGRESSION RISK:
⚠ List of existing features that could be affected by this change
```

After ANY CSS Change — Required Regression Test

Because CSS changes have repeatedly broken interactive elements, the following must be tested after **every** `globals.css` edit:

1. ✅ Click "Sign in" in nav → navigates to /login
2. ✅ Click "Get started" in nav → navigates to /signup
3. ✅ Login form submits → gets to dashboard
4. ✅ Course card is clickable
5. ✅ Primary CTA buttons respond to click

After ANY Auth-Related Change

1. ✅ Sign in with email/password
2. ✅ Sign in with Google
3. ✅ Logged-out user visiting /dashboard → redirected to /login
4. ✅ Logged-in user visiting /login → redirected to /dashboard
5. ✅ Nav shows correct state (signed in vs signed out)
6. ✅ Token expiry: session refreshes without re-login

High-Risk Areas (Always Extra Testing)

Area	Why High Risk
Auth nav buttons	CSS hover transforms have broken these multiple times
Video upload and playback	Network timeouts, transcoding failures
Checkout and enrollment	Payment succeeds but access not granted (or vice versa)
Progress tracking	Race conditions when marking lessons complete
Role-based permissions	Wrong user accessing wrong content
Proxy.ts changes	Route protection for entire app — breaks everything if wrong

Regression Trigger Table

When we change...	We always re-test...
`globals.css`	All interactive elements: nav, forms, buttons
`proxy.ts`	Every protected route, auth redirects, role-specific pages
Anything auth-related	Every protected route, enrollment access, role-specific pages
Payment or checkout	Enrollment grant, purchase history, creator sales
Video player	Progress tracking, completion state, resume position
Roles and permissions	Admin, creator, and learner access — all three
Course publish/unpublish	Browse page, direct URL, enrolled learner access

---

13. How We Work Together

Collaboration Rhythm

```
Step 1 — Before building:
  Define user story + acceptance criteria
  Confirm approach before writing code

Step 2 — During building:
  Flag any tradeoffs or decisions that need input
  Note anything that deviates from the plan

Step 3 — Testing:
  Walk through feature as a real user
  Test happy path + error path + edge cases

Step 4 — Fix and verify:
  Any failures fixed before moving forward
  Re-verify the fix

Step 5 — Regression check:
  Check Regression Trigger Table
  Update this document with what was built, any new problems, any new rules

Step 6 — Move on:
  Only after all of the above
```

Decision Making

- **Technical decisions** (how to build it, tools, DB structure) → I recommend, you approve or ask questions
- **Product decisions** (what a feature does, copy, policy) → you decide, I implement
- **Prioritization** → we decide together based on what's blocking users or revenue

What I Will Always Tell You

- When something is more complex than expected
- When a shortcut now will cause a problem later
- When a feature request could break something that already works
- When I think something should be built differently and why

What I Need From You

- Approval before starting each feature's user story
- Feedback after each testing walkthrough
- Decisions on business rules, pricing, or user-facing copy
- Access to third-party accounts when needed (Stripe, Mux, Supabase, Vercel, Resend)

---

14. Environment & Setup

Environment Variables (`.env`)

```
# Database
DATABASE_URL=                    # Supabase PostgreSQL connection string
```

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=       # Only for server-side admin operations

# Stripe
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=

# Mux
MUX_TOKEN_ID=
MUX_TOKEN_SECRET=

# Cloudflare R2
CLOUDFLARE_ACCOUNT_ID=
CLOUDFLARE_R2_BUCKET=
CLOUDFLARE_R2_ACCESS_KEY_ID=
CLOUDFLARE_R2_SECRET_ACCESS_KEY=
CLOUDFLARE_R2_PUBLIC_URL=

# Resend
RESEND_API_KEY=
```

Standing Test Accounts (do not delete)

Created 2026-06-11. Used for ongoing testing across all roles — keep these accounts permanently rather than creating new ones each session.

Account	Email	Password	Role
Test Learner	test-learner@reacademy.test	TestPass123!	LEARNER
Test Creator	test-creator@reacademy.test	TestPass123!	CREATOR
Admin (Alyssa)	alyssa.specht@exprealty.net	TestPass123!	ADMIN

Note: `SUPABASE_SERVICE_ROLE_KEY` is not currently set in `.env`. New signups require manual email confirmation in Supabase Dashboard → Authentication → Users until this is added (or "Confirm email" is disabled for the project).

Local Development

```
# Prerequisites: Node 18+, npm

# Install dependencies
npm install

# Start dev server (port 3000)
npm run dev
```

```
# Kill hung dev server
pkill -f "next dev"

# Push schema changes to database
npx prisma db push

# View database in browser
npx prisma studio
```

---

15. Pending Items & Next Up

Immediate Priorities (Auth Fix Already Done)

1. **Verify auth fix works** — have user test sign in / sign up / nav buttons in their browser
2. **Email notifications** — wire Resend templates for purchase receipts and enrollment confirmations
3. **AI Lesson Summaries** (Feature 4.5) — **blocked until video transcripts are available.** Claude cannot summarize a video with no transcript. Need to either: (a) require text transcript upload when adding a video lesson, or (b) use Mux's automatic transcript API

Next Planned Features (in order)

1. Advanced Course Search & Filters (3.3)
2. Email notifications via Resend (2.19)
3. Completion Certificates (3.5)
4. AI Lesson Summaries when transcript problem is solved (4.5)

Known Technical Debt

- Sentry not installed (0.10) — errors are invisible in production
- No automated test suite — all testing is manual
- Resend email templates not wired to all trigger events
- `getCurrentUser()` returns null if DB user doesn't exist even with valid Supabase session — no graceful handling for this edge case

Open Product Questions

- What is the platform called? (Currently "RE Academy" as placeholder)
- What is the revenue share % for creators?
- Do learners need to verify their real estate license to access content?
- What states/markets are we targeting first?

---

16. Features At a Glance (Quick Reference)

A fast scan of everything in the product — what's working, what phase it belongs to, and what's still pending. Use this list to decide what to test next.

✅ Done & Verified

Feature	Phase
Sign up / sign in / sign out (Supabase Auth + dropdown menu in nav)	1
Role-based access (LEARNER / CREATOR / ADMIN) via `proxy.ts`	1

Course creation, editing, modules & lessons (video/text/resource)	1–2
Course catalog browsing & category filters	2
Course purchase & checkout (Stripe)	2
Enrollment & lesson progress tracking	2
Course reviews & ratings	2
Coupons (percent/fixed discounts)	2
Learning Paths (bundled course sequences)	2–3
Community discussions per course (threads + replies)	2
Notifications (e.g. discussion replies)	2
Creator application & approval flow	2
Gamification: XP, streaks, badges	4
Templates/Tools Marketplace (creator + platform items, FILE/LINK delivery)	4 (bonus)
Team Roster — creator/admin adds team members by pasted email list	5.4
Course Assignment — assign one or more courses to one or more team members, auto-enrolls them	5.5
Assignment Progress Tracking — view each member's completion (X/Y lessons) per assigned course	5.5
Course Visibility (Public/Private) — creators choose Public (catalog-listed) or Private (invite-only); `/courses` only shows PUBLIC + PUBLISHED	5.5a

⏳ Not Yet Built

Feature	Phase
Advanced course search & filters (keyword search, sort, multi-filter)	3.3
Email notifications via Resend (purchase receipts, enrollment confirmations, assignment notices)	2.19
Completion certificates (PDF generation)	3.5
AI Lesson Summaries — **blocked**: requires video transcripts (not yet captured)	4.5
Sentry / error monitoring	0.10
Automated test suite	—
`SUPABASE_SERVICE_ROLE_KEY` setup (currently requires manual email confirmation in Supabase dashboard for new signups)	—

---

17. Testing Notes Template

Copy this block for each test session. Fill in as you go — doesn't need to be fancy, just enough to capture what happened so we can fix it later.

```
### Test Session — [DATE]

**Tester:** [your name / account used, e.g. test-learner@...]
**Feature(s) tested:** [e.g. Team Roster — bulk add members]

---

**Test 1: [short description, e.g. "Add 3 team members via pasted list"]**
- Steps:
  1.
  2.
  3.
- Expected:
- Actual:
- Result: ✅ Pass / ❌ Fail / ⚠️ Partial
- Notes:

---

**Test 2: [...]**
- Steps:
- Expected:
- Actual:
- Result:
- Notes:

---

**General notes / ideas for later:**
-
```

Document version 2.1 — Updated 2026-06-11 Previous version: 2.0 (Updated 2026-06-08), 1.0 (Created 2026-06-04) Platform: Real Estate Education LMS & Marketplace

> ***The Golden Rule:*** *The goal is not to build fast. The goal is to build something that works reliably and gets better over time. Every feature we verify completely is a feature we never have to come back and fix later.*